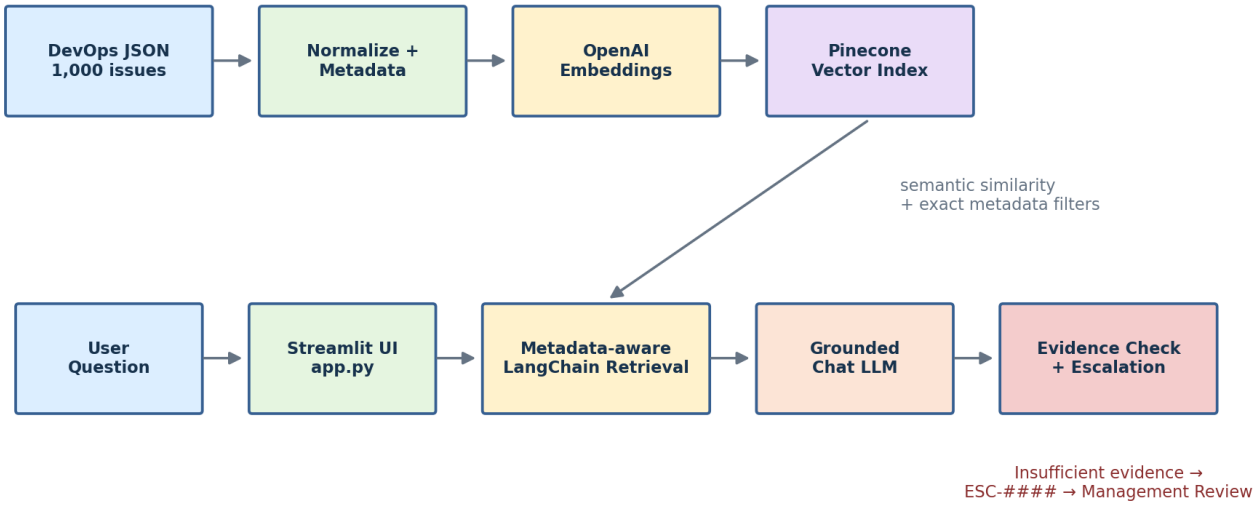
POWEROPS v1

AI-Powered DevOps Management Assistant

RAG • Metadata-Aware Search • Evidence-Based Human Escalation

PowerOps v1 — Solution Architecture



Purpose

A shareable explanation of how PowerOps converts DevOps issue data into grounded answers while routing uncertain questions to management instead of guessing.

1. Executive Overview

PowerOps v1 is a Retrieval-Augmented Generation (RAG) application that lets users query DevOps management data in natural language. It combines exact operational filtering with semantic retrieval, then asks a language model to answer only from retrieved issue evidence.

Design principle

Use deterministic metadata filters for team, assignee, priority, status and issue key; use vector similarity for meaning; use the LLM for synthesis; escalate when the evidence is insufficient.

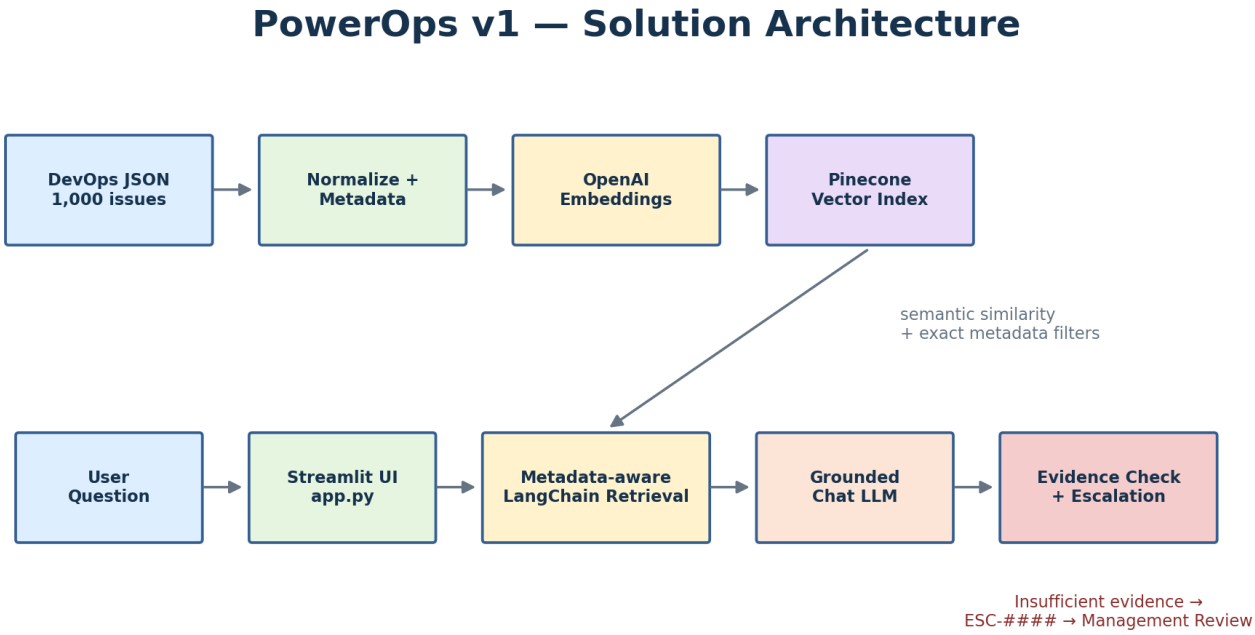
What is implemented

- 1,000 normalized DevOps issues across three teams and 62 assignees.
- Six notebooks covering data exploration, document preparation, Pinecone ingestion, metadata retrieval, RAG answering and human escalation.
- A Streamlit application that displays answers, source issue keys, parsed filters, retrieved records and escalation status.
- OpenAI text-embedding-3-small embeddings (1,536 dimensions), a Pinecone index, and a configurable OpenAI chat model.
- Default retrieval depth TOP_K=5.

Dataset snapshot

Dimension	Current project data
Assigned teams	Falcon Squad: 365, Nova Team: 438, Summit Crew: 197
Priority	Medium: 879, High: 91, Low: 3, Urgent: 27
Status	In Progress: 13, Done: 888, Rejected: 22, To Do: 22, Soft Delete: 53, On Hold: 2
Assignees	62 unique assignee values
Prepared documents	1,000 documents — one per issue

2. Solution Architecture

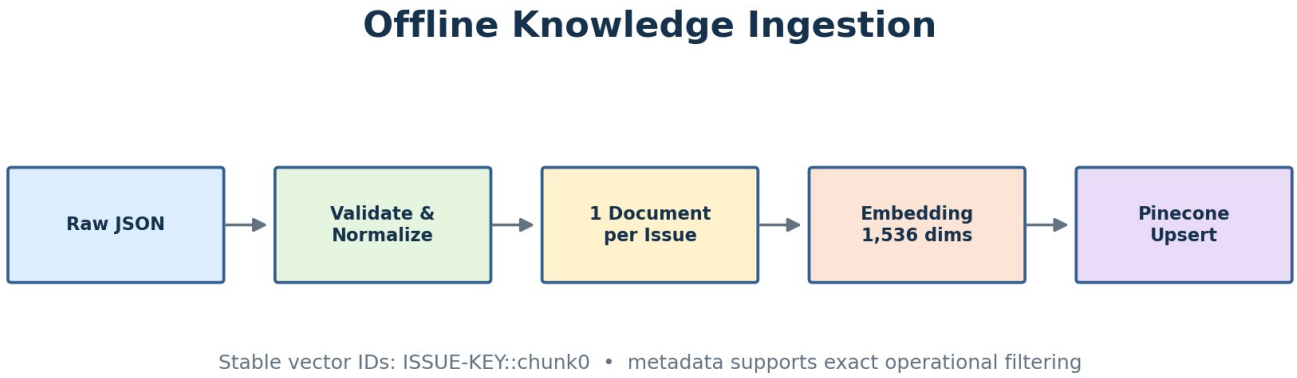


The upper path builds the searchable knowledge base. The lower path handles live user questions. Pinecone connects the two: it stores the embedded issue documents and their structured metadata, then returns the most relevant records at query time.

Component responsibilities

Component	Role	Value
Streamlit	User interface and result presentation	Simple operational access and transparency
LangChain	Orchestrates embeddings, retrieval and LLM calls	Keeps the RAG pipeline modular
OpenAI Embeddings	Turns issue text into semantic vectors	Finds related records beyond exact keywords
Pinecone	Vector search plus metadata filtering	Combines semantic relevance with operational precision
Chat LLM	Synthesizes an answer from retrieved context	Produces readable, grounded responses
Evidence checks	Validate retrieval and generated citations	Reduces unsupported answers
Escalation log	Persists ESC-#### records to JSON	Creates a human-review path

3. Data Ingestion & Knowledge Preparation



- Notebook 01 validates and normalizes the exported JSON, checks duplicates and missing values, and persists normalized_issues.json.
- Notebook 02 creates readable page_content plus metadata including issue key, team, assignee, reporter, priority, status, story points and dates.
- The current dataset uses one document per issue because each record is compact; chunk_id and chunk_count keep the design future-ready.
- Notebook 03 creates embeddings, verifies dimensions, connects to Pinecone and performs idempotent upserts using stable IDs such as INO-21920::chunk0.
- The prepared document store contains 1,000 issue documents.

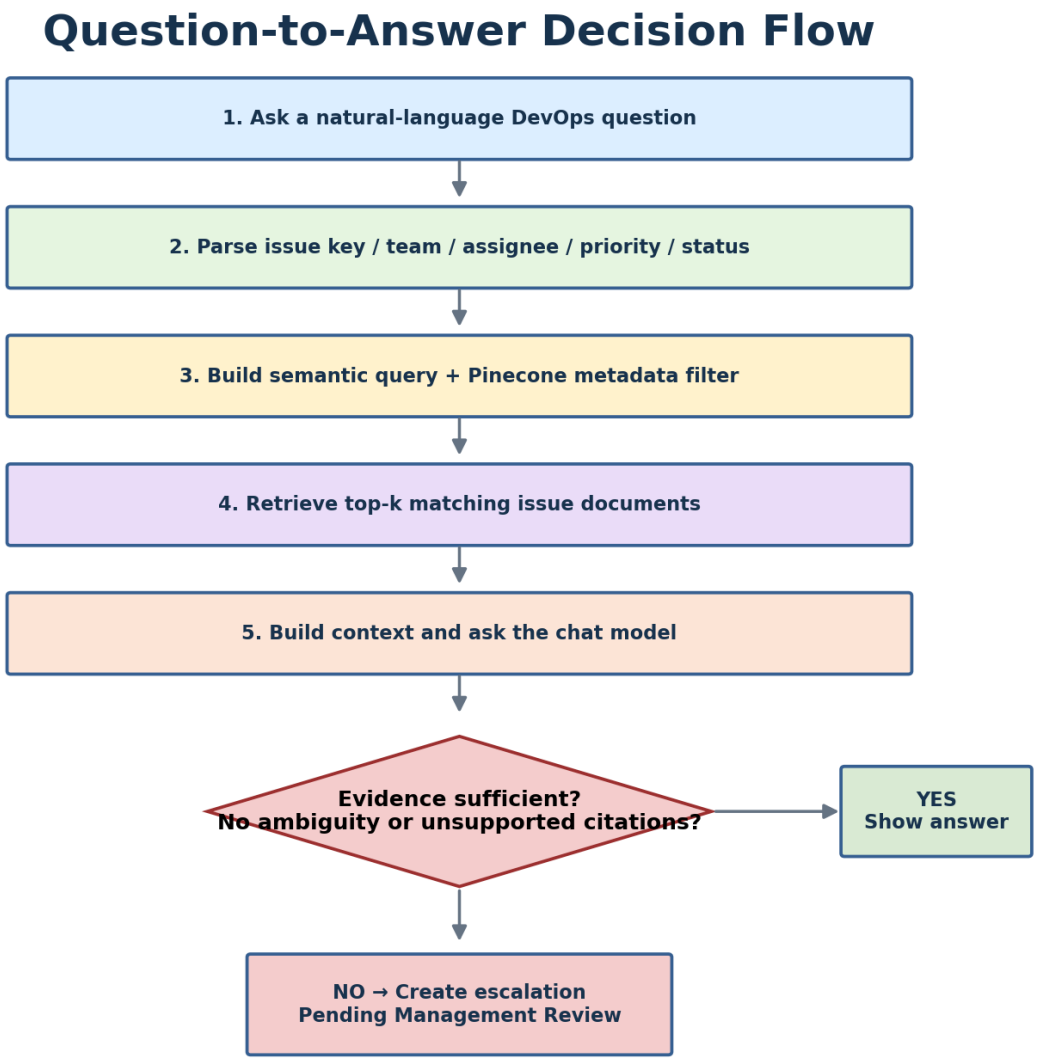
Why metadata matters

A question such as “high-priority Falcon Squad issues” should not rely only on semantic similarity. PowerOps can apply exact Pinecone filters for assigned_team=Falcon Squad and priority=High, then rank only the eligible records semantically.

Representative document

Field	Example
Issue key	INO-21920
Assigned team	Falcon Squad
Assignee	Sullivan, Deepa (Contractor)
Priority	Medium
Status	In Progress
Vector ID	INO-21920::chunk0

4. Query, Retrieval & Answer Flow



The retrieval layer first detects structured intent in the question. It recognizes issue keys and vocabulary values for team, priority and status, and applies special handling for assignee names because partial names can be ambiguous.

- parse_query_filters() separates structured filters from the semantic portion of the question.
- build_pinecone_filter() translates recognized fields into exact $eq metadata constraints.
- similarity_search_with_score() retrieves the top matching issue documents.
- build_context() formats retrieved records into evidence blocks.
- The chat model receives a system instruction plus the PowerOps context and the original question.
- The answer is returned with source issue keys and retrieval details so users can inspect what supported it.

Example questions

Question	Expected behavior
What is the status of INO-21920?	Exact issue-key filter, retrieve the issue, answer from that record.

Show high-priority issues for Falcon Squad.	Apply team + priority filters, then semantic ranking.
What is Deepa working on?	Resolve assignee; if the name maps uniquely, filter and retrieve.
What is blocking the Phoenix team?	If Phoenix is absent from the vocabulary/evidence, retrieval may fail and escalation is appropriate.

5. Evidence-Based Human Escalation

PowerOps does not simply trust a model-generated confidence score. The implementation evaluates concrete evidence signals after retrieval and answer generation. If any configured failure condition is present, the request is escalated.

Escalation signal	Meaning
Zero results	No relevant PowerOps records were retrieved.
Requested issue key missing	The user named an issue that is not in retrieved evidence.
Requested filters not reflected	No retrieved record satisfies the requested team/assignee/priority/status combination.
Ambiguous assignee	A partial name matches multiple people.
LLM declined	The model explicitly indicates the context is insufficient.
Unsupported citations	The answer mentions an issue key not present in retrieved evidence.

Escalation record

When escalation is required, PowerOps creates an ID such as ESC-0001 and records timestamp, question, reasons, retrieved issue IDs, evidence details and status = “Pending Management Review”.

Why this is important

This pattern is stronger than a chatbot that always answers. In operational management, an explicit “I do not have enough evidence” path is often safer and more useful than a fluent but unsupported response.

6. Technology Stack & Project Map

Technology	Current use
Python	Core application and notebooks
Streamlit	Interactive web UI
LangChain	RAG orchestration and vector-store integration
Pinecone	Vector database and metadata-filtered retrieval
OpenAI text-embedding-3-small	1,536-dimensional embeddings
OpenAI chat model	Grounded response generation; configured in .env
JSON	Source data, normalized/prepared artifacts and escalation persistence

Notebook progression

Notebook	Purpose
01_data_loading_and_exploration	Load, inspect, normalize and validate source data
02_document_preparation	Create issue documents and metadata
03_pinecone_ingestion	Embed and upsert documents into Pinecone
04_metadata_retrieval	Parse structured filters and test retrieval
05_powerops_rag	Build the grounded question-answering workflow
06_human_escalation	Add deterministic evidence checks and escalation

Runtime application

app.py consolidates the retrieval, grounding and escalation logic into the Streamlit experience. It loads vocabulary, connects to Pinecone and OpenAI, parses questions, retrieves records, invokes the model, evaluates evidence and persists escalations.

7. Strengths, Limitations & Recommended Next Steps

Current strengths

- Clear separation between structured filtering and semantic search.
- Traceable answers with issue-key sources and retrieval details.
- Deterministic escalation checks rather than model confidence alone.
- Idempotent vector IDs and a notebook-by-notebook implementation path that is easy to explain and test.
- Configuration is externalized through environment variables rather than hard-coded credentials.

Current limitations

- The knowledge base is a point-in-time JSON export; there is no automated source-system synchronization in the current project.
- Escalations are persisted to a local JSON file rather than a workflow/ticketing platform.
- The Streamlit app is a prototype interface; enterprise authentication, authorization and audit controls are not shown.
- Evidence checks validate several important failure modes, but similarity score thresholds and formal retrieval-quality evaluation could be expanded.
- The current corpus is compact and uses one document per issue; larger ticket histories or attachments would require a richer chunking strategy.

Recommended roadmap

Priority	Enhancement	Outcome
1	Connect to the authoritative DevOps/Jira/ITSM source via scheduled or event-driven ingestion	Fresher operational knowledge
2	Route escalations into email, Teams/Slack, Jira or ServiceNow with owner and SLA	Closed-loop human review
3	Add SSO/RBAC and team-level data permissions	Enterprise-ready access control
4	Build an evaluation set for retrieval precision, groundedness and escalation accuracy	Measurable quality
5	Add management dashboards for open issues, aging, priority and escalation trends	Move from Q&A to decision support
6	Capture human resolutions and feed approved answers back into the knowledge workflow	Continuous improvement

Target outcome

PowerOps can evolve from a RAG prototype into an operational decision-support assistant: searchable, explainable, governed, and connected to the systems where DevOps work actually happens.

8. Summary

PowerOps v1 demonstrates a practical RAG pattern for DevOps management data. Structured metadata provides precision, vector search provides semantic flexibility, the LLM provides readable synthesis, and evidence-based escalation provides a safety net. The result is a system that can answer routine operational questions quickly while making uncertainty visible and actionable.